

16 Lecture 16: Complexity

16.1 Asymptotic notation

A notation that captures the essential behaviour of a function for large values of its variable.

1) $f(n) = O(g(n))$ if there exists a n_0 such that for any $n > n_0$, $f(n) \leq c g(n)$ with $c = \text{constant}$.

2) $f(n) = \Omega(g(n))$ if there exists a n_0 such that for any $n > n_0$, $f(n) \geq c g(n)$ with $c = \text{constant}$.

3) $f(n) = \Theta(g(n))$ if $f(n)$ and $g(n)$ have the same asymptotic behaviour up to a constant factor. If $f(n) = O(g(n)) = \Omega(g(n))$ then $f(n) = \Theta(g(n))$.

Examples:

- $2n$ is $O(n^2)$, 2^n is $\Omega(n^3)$
- $7n^2 + \sqrt{n} \log n$ is $\Theta(n^2)$ since $7n^2 \leq 7n^2 + \sqrt{n} \log n \leq 8n^2$ for n sufficiently large.
- $f(n) = O(g(n)) \Leftrightarrow g(n) = \Omega(f(n))$, $f(n) = \Theta(g(n)) \Leftrightarrow g(n) = \Theta(f(n))$.
- if $g(n)$ is a polynomial of order k , $g(n) = O(n^l)$ for $k \leq l$.
- $\log n = O(n^k)$ for any $k > 0$.
- $n^{\log n}$ is superpolynomial: $n^k = O(n^{\log n})$ for any k , but $n^{\log n} \neq O(n^k)$.
- $n^{\log n}$ is subexponential: $c^n = \Omega(n^{\log n})$ for any $c > 1$, but $n^{\log n} \neq \Omega(c^n)$.
- if $e(n) = O(f(n))$ and $g(n) = O(h(n))$, then $e(n)g(n) = O(f(n)h(n))$.

Exercise: : show that ordering n names alphabetically is $\Omega(n \log n)$. Since algorithms are known, that are $O(n \log n)$, the ordering problem is $\Theta(n \log n)$.

16.2 Computational complexity

Easy problems: solution can be found in polynomial time, i.e. with a number of steps that is a polynomial function of the dimension n of the input. For example the addition of two n -bit integers scales as n .

Hard problems: superpolynomial in n . For example the factorization of an integer N requires a number of steps scaling as $\exp n$, if n is the number of digits of N .

The main *complexity classes* are:

P: problems that can be solved in polynomial time (easy problems).

NP: problems whose solution can be verified in polynomial time.

NP-complete: any NP problem can be reduced to a NP-complete problem.

Obviously NP includes P. No proof exists, but it is generally believed that **NP** $\neq$ **P**. If a polynomial-time resolution of a NP-complete problem was ever found, any NP problem could be solved in polynomial time, which would imply $P = NP$. The current consensus is that no polynomial time resolution of a NP-complete problem exists.

For example integer factorization is **NP**, since the solution can be checked (by multiplication) in polynomial time. However no polynomial time factorization algorithm is known within classical computation. The situation changes with quantum computation: the Shor algorithm, based on the quantum Fourier transform (see Lecture 23), factorizes in polynomial time. Unfortunately the factorizing problem is not NP-complete, so that we cannot conclude $P = NP$ within quantum computation.

16.3 Examples

- A *graph* is a finite collection of *vertices* $\{v_1, \dots, v_n\}$, connected by links (v_i, v_j) . The graph is non-directional if the order of the vertices in the links is irrelevant. A *cycle* is a sequence of vertices $\{v_1, \dots, v_m\}$ such that every pair (v_j, v_{j+1}) is a link and (v_m, v_1) is a link. In words, a cycle is a closed path in the graph.

A cycle is *simple* if all its vertices appear only once.

A cycle is *Hamiltonian* if it is simple and includes all the vertices of the graph.

A cycle is *Eulerian* if every link of the graph is “visited” only once.

Given a graph, no polynomial algorithm is known to decide whether it admits a Hamiltonian cycle. Moreover the Hamiltonian cycle (HC) problem can be shown to be NP-complete.

Perhaps surprisingly, the Eulerian cycle (EC) problem is in class P. In fact, a graph admits an Eulerian cycle if it is connected and only an even number of lines originate from every vertex of the graph, a result due to Euler. Checking that it is connected requires $O(n^2)$ operations, and checking the number of incident lines for every vertex requires $O(n^3)$ operations (there are at most $n(n-1)/2$ lines in the graph with n vertices). Thus EC is in class P.

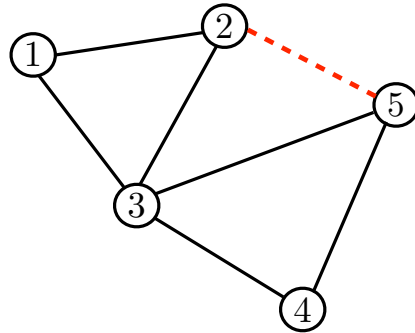


Fig. 16.1 Without the dashed line, this graph has no Hamiltonian cycle, but has a Eulerian cycle. With the dashed line, there is a Hamiltonian cycle, but no Eulerian cycle.

Both HC and EC are examples of *decisional problems*: an algorithm solving the problem answers yes or no to the question of existence of particular cycles.

- The Traveling Salesman Problem (TSP) is another NP-complete decisional problem, and is formulated as follows:

“Given n towns and a distance d_{ij} between town i and town j , is there a closed path including all towns with total length $< d$? ”

We now prove that HC reduces to TSP. It suffices to apply TSP with the following specifications: if the graph has n vertices, each vertex is considered a town, and the distance d_{ij} between towns is assigned to be 1 if the corresponding vertices in the graph are connected by a link, and 2 otherwise. Then with TSP we can answer the question: does a closed path connecting all towns exist, with length $< n + 1$?. An algorithm solving TSP will provide the answer: if positive, it implies that such a path exists, and its length is n (it must be $< n + 1$ and, since the closed path connects n towns, it must be at least n). Then a Hamiltonian cycle exists in the graph, including all its vertices only once. $\square$

Thus, given an algorithm that solves TSP, it can be converted to an algorithm that solves HC. Since HC is NP-complete, this implies that also TSP is NP-complete: if every NP problem reduces to HC, it also reduces to TSP.

- CSAT (circuit satisfiability) is the decisional problem: “Given a boolean circuit composed by AND, OR, NOT gates, does an input assignment exist that gives as output the bit 1 ? ”. This problem can be shown (Cook-Levin proof) to be NP-complete, by use of the Turing machine paradigm, see for ex. Chuang and Nielsen, p.146. Thus if a problem is NP, and CSAT reduces to it, the problem is also NP-complete. In this way many other problems have been shown to be NP-complete.

- SUBSET SUM is another NP-complete problem, well suited to be attacked also by quantum computing protocols. It consists in finding whether a subset exists, in a list of integers $(n_1, \dots, n_N)$, having as sum a given integer S .

Note: no quantum protocol is known to solve in polynomial time NP-complete problems. It is widely believed that $P \neq NP$ also within quantum computation.

16.4 Energy and information

Energy requirements for computation are related to *reversibility*. In irreversible gates, as for example the classical NAND gate, there is loss of information. To cancel information, energy is required, as stated in Landauer's principle (1961):

To cancel 1 bit, an energy of at least $kT \log 2$ is required.

This principle can be justified by considering the encoding of 1 bit by using a gas in a box. The gas is very peculiar, consisting of only one molecule. The box is divided into two parts, with a mobile partition. As such, the box contains 1 bit of information, which has value 0 if the molecule is on left of the partition, and value 1 if on the right of the partition. Suppose we want to cancel this information: we remove the partition and compress the gas to the left with a piston, which slides halfway into the box. Now the molecule will be located with certainty in the left part of the box, and carries no information. A bit has been cancelled. But work was necessary to compress the gas. This work can be computed by considering the entropy of the system before and after compression. In the initial state, the box had entropy $S = k \log(\text{number of microstates}) = k \log 2$, since there are two possible microstates (molecule on the left, molecule on the right). After the compression, only 1 microstate survives, and the box has entropy $= 0$. Thus the box entropy has decreased by $\Delta S = k \log 2$, which corresponds to a work $W = T\Delta S = kT \log 2$.

16.5 Reversible classical computation: the TOFFOLI gate

The gates for a classical universal set, NAND and FANOUT, are non reversible. Information is lost: from the output we cannot reconstruct the input. Nevertheless, it is possible to simulate NAND and FANOUT with reversible gates. For example the Toffoli gate, a 3-bit to 3-bit gate, can be used as in Fig. 16.2 to act as NAND or FANOUT. Thus Toffoli by itself can be considered a universal set.

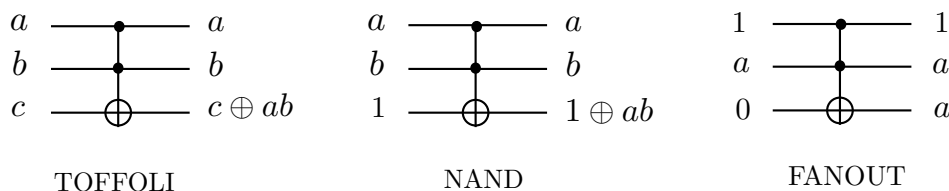


Fig. 16.2 The TOFFOLI gate, and its use to simulate NAND and FANOUT

This shows that classical computation can be made reversible, but at the cost of introducing service bits in input (“ancilla bits”), and producing extra bits in

output (“garbage bits”). These extra bits must be cancelled at some stage, to reduce cluttering of the memory, and will require energy for their cancellation.